

[Get started](#) [Prepare](#)[Get full access](#)[Blog Menu](#)[Series Contents](#)

Angular Interview Questions for Experienced

Master advanced Angular interview questions for experienced developers. 25+ expert-level questions covering RxJS, testing, performance optimization, and real-world scenarios for senior roles.

[Advanced](#)[#angular](#)[#interviews](#)**GreatFrontEnd Team**

29 min read

Oct 15, 2025

[Copy link](#)[Share](#)

Angular Interview Questions for Experienced

If you're an experienced frontend developer preparing for your next Angular interview, this post is for you. With the evolving features in Angular's ecosystem - from standalone components to Signals, hydration, and zone-less change detection - interviewers now expect deep architectural understanding.

This post is part of our comprehensive [Angular Interview Questions and Answers Guide](#). Here, we'll cover the most asked Angular Interview Questions for Experienced Professionals, including **advanced**, **scenario-based**, and **performance-related** topics.

Why read this post?

If you classify yourself as:

- A **senior Angular developer** preparing for product-based interviews
- A **frontend engineer** targeting enterprise-level architecture roles

- Or a **tech lead** revising your fundamentals before mentoring others

then this list of Angular interview questions for experienced professionals will be your ultimate preparation resource.

Below is a curated list of 25 questions that target key areas - architecture, DI internals, change detection, RxJS, and performance optimization.

1. Explain the MVVM architecture pattern in Angular.

Angular follows **Model-View-ViewModel (MVVM)** pattern where:

- **Model:** Data and business logic (services, HTTP calls)
- **View:** Template (HTML)
- **ViewModel:** Component class that binds data to view

```
1  // ViewModel (Component)
2  export class UserComponent {
3      users$ = this.userService.getUsers(); // Model interaction
4
5      constructor(private userService: UserService) {}
6
7      deleteUser(id: string) {
8          this.userService.deleteUser(id).subscribe();
9      }
10 }
```

The component acts as the ViewModel, managing state and handling user interactions while the template renders the view.

2. What is an Angular NgModule, and how does it organize code structure?

An **NgModule** in Angular is a container that groups related code-components, directives, pipes, and services-into a cohesive block of functionality. It helps organize the application into logical modules for better maintainability and reusability.

Each Angular app has at least one root module (`AppModule`), which bootstraps the app. Other feature modules can be created to encapsulate specific features or functionalities.

```
1  import { NgModule } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3  import { AppComponent } from './app.component';
4
5  @NgModule({
6    declarations: [AppComponent], // Components, directives, pipes
7    imports: [BrowserModule], // Other modules
8    providers: [], // Services
9    bootstrap: [AppComponent], // Root component
10 })
11 export class AppModule {}
```

Note: From Angular 14 onward, standalone components can be used without NgModules, but NgModules remain useful for grouping imports and providing services in larger apps.

3. Describe the Angular bootstrapping process and what happens internally.

Angular bootstrapping process:

1. **main.ts** calls `platformBrowserDynamic().bootstrapModule(AppModule)`
2. Angular creates the **platform** and **application injector**
3. **AppModule** is instantiated and its providers are registered
4. **Bootstrap component** (usually `AppComponent`) is created
5. Component is **rendered** into the DOM at the specified selector

// main.ts

```
platformBrowserDynamic()  
  .bootstrapModule(AppModule)  
  .catch((err) => console.error(err));
```

4. How does the `inject()` API improve dependency injection compared to constructor-based DI?

The `inject()` API, introduced in Angular 14, allows dependencies to be injected outside of class constructors, such as inside functions, factory providers, or standalone components. It provides more flexibility and cleaner code compared to constructor-based DI.

Benefits:

- Works in functional contexts (not limited to classes)
- Simplifies testing and reusability
- Enables dependency injection in providers without creating extra classes

```
1  // Traditional constructor injection  
2  export class UserComponent {  
3    constructor(private userService: UserService) {}  
4  }  
5  
6  // Using inject() API  
7  export class UserComponent {  
8    private userService = inject(UserService);  
9  
10 // Can be used in functions  
11 loadUsers = () => {  
12   const http = inject(HttpClient);  
13   return http.get('/api/users');  
14 };  
15 }
```

5. Explain the importance of OnPush change detection and how to apply it effectively.

The **OnPush** change detection strategy improves performance by limiting when Angular checks for changes. Instead of running on every event, it only triggers when:

- An input property changes (by reference)
- An event originates from the component or its children
- You manually mark it for check (`ChangeDetectorRef.markForCheck()`)

This makes components more predictable and faster, especially in large apps.

How to use: Apply it in the component decorator:

```
1  @Component({
2    changeDetection: ChangeDetectionStrategy.OnPush,
3    template: `<div>{{ user.name }}</div>`,
4  })
5  export class UserComponent {
6    @Input() user: User;
7
8    constructor(private cdr: ChangeDetectorRef) {}
9
10   updateUser() {
11     // Use immutable updates
12     this.user = { ...this.user, name: 'Updated' };
13     this.cdr.detectChanges(); // Manual trigger if needed
14   }
15 }
```

Tip: Always pass new object references (`{...obj}` or arrays via spread) to trigger updates in OnPush components.

6. How would you implement lazy loading, and when is it useful?

Lazy loading in Angular means loading feature modules or components only when needed, rather than at app startup. It improves performance by reducing the initial bundle size and speeding up load times - especially useful for large apps with multiple routes.

How to implement (module-based):

1. Create a feature module (e.g., `users.module.ts`)
2. Configure a route using `loadChildren` :

```
1  const routes: Routes = [  
2    {  
3      path: 'users',  
4      loadChildren: () =>  
5        import('./users/users.module').then((m) => m.UsersModule),  
6    },  
7  ];
```

Standalone component example (Angular 15+):

```
1  const routes: Routes = [  
2    {  
3      path: 'profile',  
4      loadComponent: () =>  
5        import('./profile/profile.component').then((c) => c.ProfileComponent),  
6    },  
7  ];
```

Use it when: Your app has multiple large sections that aren't always visited (e.g., admin dashboard, reports, settings).

7. What is the difference between providers and viewProviders ?

Both `providers` and `viewProviders` define services that a component and its children can use - but they differ in scope.

- `providers` - Makes the service available to the component and all its content children (including projected components via `<ng-content>`)

- `viewProviders` - Limits the service to the component's view only (excludes projected content)

```
1  @Component({
2    selector: 'app-parent',
3    template: `<ng-content></ng-content>`,
4    providers: [LoggerService], // Available to projected children
5    viewProviders: [AuthService], // Only for this component's view
6  })
7  export class ParentComponent {}
```

8. Describe the process and benefits of Ahead-of-Time (AOT) Compilation.

Ahead-of-Time (AOT) Compilation is the process of compiling Angular templates and TypeScript code **at build time**, before the application runs in the browser.

How it works

1. Angular CLI compiles templates, metadata, and decorators into optimized JavaScript during the build
2. The browser loads **pre-compiled code**, eliminating the need for runtime compilation

Benefits

- **Faster startup:** Templates are pre-compiled, reducing browser work.
- **Early error detection:** Template and type errors are caught at build time
- **Smaller bundles:** Removes Angular compiler from the final code
- **Improved security:** Reduces risks of injection attacks in templates

Angular CLI uses AOT by default in production builds:

```
ng build --configuration production
```

9. How do you create and manage custom directives (structural/attribute)?

Directives in Angular are classes that add behavior or modify the DOM. There are two main types:

1. **Attribute Directives** - Change the appearance or behavior of an element
2. **Structural Directives** - Change the DOM structure by adding or removing elements

1. Creating an Attribute Directive

```
1  import { Directive, ElementRef, Renderer2, HostListener } from '@angular/core';
2
3  @Directive({
4    selector: '[appHighlight]'
5  })
6  export class HighlightDirective {
7    constructor(private el: ElementRef, private renderer: Renderer2) {}
8
9    @HostListener('mouseenter') onMouseEnter() {
10      this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'yellow');
11    }
12
13    @HostListener('mouseleave') onMouseLeave() {
14      this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'transparent');
15    }
16  }
17  **Usage:**
18  ```html
19  <p appHighlight>Hover over me!</p>
```

2. Creating a Structural Directive

Structural directives use `*` syntax and manipulate the DOM.

```

1  import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';
2
3  @Directive({
4    selector: '[appUnless]',
5  })
6  export class UnlessDirective {
7    constructor(
8      private templateRef: TemplateRef<any>,
9      private vcRef: ViewContainerRef,
10   ) {}
11
12   @Input() set appUnless(condition: boolean) {
13     if (!condition) {
14       this.vcRef.createEmbeddedView(this.templateRef);
15     } else {
16       this.vcRef.clear();
17     }
18   }
19 }

```

Usage:

```
<p *appUnless="isVisible">I am visible only when isVisible is false</p>
```

Managing Directives

- Declare them in an NgModule (or use standalone components/directives in Angular 14+)
- **Scope control:** Apply only to specific components by including in module declarations
- **Reuse:** Combine with `@Input` and `@Output` to make directives flexible and configurable

10. Compare Signals and RxJS Observables - when would you use each?

Signals and **Observables** both handle reactive data in Angular but differ in scope and use case.

Signals

- Angular 16 feature for **local reactive state**
- Updates the UI automatically when the value changes
- Best for **component-level state**

```
import { signal } from '@angular/core';
```

```
const count = signal(0);
```

```
count.set(count() + 1);
```

RxJS Observables

- Streams of data over time (async operations, events)
- Powerful operators for **filtering, mapping, combining**
- Best for **HTTP calls, events, or shared state**

11. How does `Renderer2` work, and why is direct DOM access discouraged in Angular?

`Renderer2` is an Angular service that provides **safe, platform-independent methods** to manipulate the DOM (create elements, set styles, listen to events) without touching the browser APIs directly.

Why use `Renderer2`

- Ensures **cross-platform compatibility** (browser, server-side rendering, Web Workers)
- Helps **prevent XSS attacks** by sanitizing changes
- Makes **unit testing easier**, since DOM operations can be mocked

Example: Using `Renderer2`

```
1  import { Directive, ElementRef, Renderer2, HostListener } from '@angular/core';
2
3  @Directive({
4    selector: '[appHighlight]',
5  })
6  export class HighlightDirective {
7    constructor(
8      private el: ElementRef,
9      private renderer: Renderer2,
10   ) {}
11
12   @HostListener('mouseenter') onMouseEnter() {
13     this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', 'yellow');
14   }
15
16   @HostListener('mouseleave') onMouseLeave() {
17     this.renderer.setStyle(
18       this.el.nativeElement,
19       'backgroundColor',
20       'transparent',
21     );
22   }
23 }
```

Avoid direct DOM access (e.g., `document.querySelector`) because it breaks Angular's platform abstraction and may fail in SSR or web worker contexts.

12. What are tree-shakable providers, and how do they optimize bundle size?

Tree-shakable providers are services in Angular that are provided **at the root level** using `providedIn: 'root'` or in a **standalone component/service**, allowing unused services to be removed from the final bundle during build.

Benefits

- **Smaller bundle size:** Unused services are automatically excluded

- **Automatic dependency management:** No need to manually add services to NgModule providers
- **Scoped injection:** Services can also be provided at component or module level if needed

Example

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable({
4    providedIn: 'root', // Tree-shakable provider
5  })
6  export class AuthService {
7    login() {
8      /* ... */
9    }
10 }
```

Only services actually used in the app are included in the production bundle, optimizing performance and load time.

13. Explain the dependency injection hierarchy and token resolution in Angular.

Angular uses a **hierarchical DI system**: injectors are organized in a tree, and services are resolved top-down.

Hierarchy:

1. **Root injector** - singleton services (providedIn: 'root')
2. **Module injector** - services scoped to feature modules
3. **Component injector** - services provided via providers or viewProviders

Token Resolution: Angular checks the **component injector first**, then moves **up the tree** to module and root injectors. If the service is not found, an error is thrown.

```
1  @Injectable({ providedIn: 'root' })
2  export class ApiService {}
3
4  @Component({
5    selector: 'app-child',
6    template: `<p>Child Component</p>`,
7    providers: [ChildService],
8  })
9  export class ChildComponent {
10    constructor(
11      private api: ApiService,
12      private childService: ChildService,
13    ) {}
14  }
```

ApiService → root injector, ChildService → component injector.

14. What are resolution modifiers (Optional , Self , SkipSelf), and how are they used?

Resolution modifiers control how Angular resolves dependencies in the injector hierarchy.

1. @Optional()

- Marks a dependency as optional
- If the service is not found, Angular injects `null` instead of throwing an error

```
constructor(@Optional() private logger?: LoggerService) {}
```

2. @Self()

- Tells Angular to look only in the current injector
- Throws an error if the service is not found locally

```
constructor(@Self() private localService: LocalService) {}
```

3. @SkipSelf()

- Skips the current injector and looks only in parent injectors
- Useful to avoid using a service provided at the current component level

```
constructor(@SkipSelf() private parentService: ParentService) {}
```

15. How can you share data between distant components in a large application?

In large Angular applications, **distant components** (not parent-child) can share data using services with observables or signals.

1. Using a Shared Service with RxJS

```
1  import { Injectable } from '@angular/core';
2  import { BehaviorSubject } from 'rxjs';
3
4  @Injectable({ providedIn: 'root' })
5  export class SharedService {
6    private messageSource = new BehaviorSubject<string>('Hello');
7    currentMessage$ = this.messageSource.asObservable();
8
9    updateMessage(msg: string) {
10      this.messageSource.next(msg);
11    }
12  }
```

Component A (sender):

```
this.sharedService.updateMessage('New Message');
```

Component B (receiver):

```
this.sharedService.currentMessage$.subscribe((msg) => console.log(msg));
```

2. Using Signals (Angular 16+)

```
1  import { signal, effect } from '@angular/core';
2
3  export const sharedSignal = signal('Hello');
4
5  // Component A
6  sharedSignal.set('New Message');
7
8  // Component B
9  effect(() => {
10     console.log(sharedSignal());
11 });
```

16. What is the purpose of an InjectionToken , and when would you use it?

An `InjectionToken` is a unique token used to inject **non-class values** (e.g., strings, objects) into Angular's DI system.

Example:

```
1  import { InjectionToken, Inject } from '@angular/core';
2
3  export const API_URL = new InjectionToken<string>('API_URL');
4
```

```
5  @NgModule({
6    providers: [{ provide: API_URL, useValue: 'https://api.example.com' }],
7  })
8  export class AppModule {}
9
10 @Component({
11   /* ... */
12 })
13 export class ApiService {
14   constructor(@Inject(API_URL) private apiUrl: string) {}
15 }
```

Use for: Configuration objects, strings, functions, or any non-class dependencies.

17. How do you test components that depend on HttpClient or routing modules?

Testing Components with HttpClient or Routing

Use Angular's testing modules to mock HTTP requests and routes without real calls.

HttpClient

```
1  import {
2    HttpClientTestingModule,
3    HttpTestingController,
4  } from '@angular/common/http/testing';
5
6  TestBed.configureTestingModule({
7    imports: [HttpClientTestingModule],
8  });
```

Use HttpTestingController to mock requests and provide test data.

Routing

```
1  import { RouterTestingModule } from '@angular/router/testing';
2
3  TestBed.configureTestingModule({
4    imports: [
5      RouterTestingModule.withRoutes([{ path: 'home', component: MyComponent }]),
6    ],
7  });
```

Use `RouterTestingModule` to simulate navigation and test route-related logic.

18. What are pure vs. impure pipes? Which one is better for performance?

Pipes transform data in templates and can be **pure** or **impure**.

- **Pure Pipes** (default)
 - Run only when input reference changes
 - Better performance

```
1  @Pipe({ name: 'pureExample', pure: true })
2  export class PureExamplePipe implements PipeTransform {
3    transform(value: string) {
4      return value.toUpperCase();
5    }
6  }
```

- **Impure Pipes**
 - Run on every change detection cycle, regardless of input changes
 - Use sparingly due to performance impact

```
1  @Pipe({ name: 'impureExample', pure: false })
2  export class ImpureExamplePipe implements PipeTransform {
3    transform(value: string) {
```

```
4     return value.toUpperCase();  
5   }  
6 }
```

19. Explain the difference between structural and attribute directives.

Directives in Angular modify the DOM or element behavior. They are of two types: **structural** and **attribute**.

Structural Directives

- Change the **DOM structure** by adding or removing elements
- Use `*` syntax
- Examples: `*ngIf` , `*ngFor`

```
<p *ngIf="isVisible">Visible only when isVisible is true</p>
```

Attribute Directives

- Change the **appearance or behavior** of an element
- Examples: `ngClass` , `ngStyle` , custom directives

```
<div [ngClass]="{ active: isActive }">Content</div>
```

20. How would you identify and fix a performance bottleneck in a large Angular app?

Identifying and Fixing Performance Bottlenecks in Angular

1. Identify Bottlenecks

- Use **Chrome DevTools Performance tab** to profile rendering and scripts
- Enable **Angular DevTools** to check **change detection cycles** and component re-renders
- Look for:
 - Components updating too frequently
 - Large lists without virtualization
 - Unnecessary API calls or computations in templates

2. Fix Bottlenecks

- Use **OnPush change detection** for components
- Implement **trackBy** in `*ngFor` for large lists
- **Lazy load** modules and components
- Move heavy computations to **pure pipes** or services.
- Debounce frequent events (e.g., `input` , `scroll`)
- Optimize **RxJS streams** using operators like `shareReplay` , `take` , `debounceTime`

Example: OnPush with trackBy

```
1  @Component({
2    selector: 'app-list',
3    template: `
4      <div *ngFor="let item of items; trackBy: trackById">{{ item.name }}</div>
5    `,
6    changeDetection: ChangeDetectionStrategy.OnPush,
7  })
8  export class ListComponent {
9    @Input() items: Item[] = [];
10
11    trackById(index: number, item: Item) {
12      return item.id;
13    }
14  }
```

```
14 }
```

Regular profiling and change detection strategy adjustments help maintain smooth performance in large Angular apps.

21. What's the significance of `trackBy` in `*ngFor` , and how does it improve rendering?

`trackBy` helps Angular identify which items changed, preventing unnecessary DOM re-rendering.

```
1  @Component({
2    template: `<div *ngFor="let user of users; trackBy: trackByUserId">
3      {{ user.name }}
4    </div>`,
5  })
6  export class UserListComponent {
7    users = [
8      { id: 1, name: 'John' },
9      { id: 2, name: 'Jane' },
10   ];
11
12   // Only re-renders changed items instead of recreating all DOM nodes
13   trackByUserId(index: number, user: User): number {
14     return user.id;
15   }
16 }
```

Benefits: Reduces DOM manipulation, improves performance for large lists, preserves component state during updates.

22. Compare `combineLatest` , `withLatestFrom` , and `forkJoin` in RxJS.

These operators handle multiple observables differently:

- `combineLatest` - emits latest values from all observables whenever **any** emits

```
combineLatest([obs1, obs2]).subscribe(([a, b]) => console.log(a, b));
```

- `withLatestFrom` - emits when the source observable emits, combining it with the latest from other observables

```
obs1.pipe(withLatestFrom(obs2)).subscribe(([a, b]) => console.log(a, b));
```

- `forkJoin` - waits for all observables to complete, then emits the last values as an array

```
forkJoin([obs1, obs2]).subscribe(([a, b]) => console.log(a, b));
```

Use: `combineLatest` for real-time updates, `withLatestFrom` to combine with source events, `forkJoin` for parallel one-time operations.

23. What are Signals effects, and when should you use them?

Signal effects are reactive side-effects that run automatically whenever a signal value changes similar to `useEffect` in React. They let you respond to state changes outside the template.

Example

```
1  import { signal, effect } from '@angular/core';
2
3  const count = signal(0);
4
5  effect(() => {
6    console.log(`Count changed: ${count()}`);
7  });
8
```

```
9 count.set(1); // Triggers effect, logs "Count changed: 1"
```

When to use:

- Respond to signal changes with side-effects (e.g., logging, calling services)
- Keep component templates pure while performing reactive actions

24. How do you debug a "Expression has changed after it was checked" error?

This error occurs when Angular detects a change to a value after change detection has run.

Steps to Debug

1. **Identify the source:** Check the component/template causing the error
2. **Check lifecycle hooks:** Avoid updating bound values in `ngAfterViewInit` or `ngAfterContentInit` directly
3. **Use `setTimeout` or `Promise`:** Delay updates to the next microtask cycle if necessary

```
1 ngAfterViewInit() {  
2   setTimeout(() => {  
3     this.value = newValue; // Updates safely after change detection  
4   });  
5 }
```

4. **** Consider OnPush strategy**:** Ensures Angular only checks the component when inputs change
5. **Avoid changing inputs during rendering:** Keep bound values stable during a single change detection cycle

The key is to **update values before or after change detection**, not during.

25. What is the role of NgZone and how do you optimize applications using runOutsideAngular() ?

NgZone manages Angular's **change detection**, automatically triggering it when async tasks (e.g., timers, events) complete.

Optimizing Performance

- Use `runOutsideAngular()` to execute code **without triggering change detection**, e.g., for high-frequency events like scrolling or animations.

```
1  import { Component, NgZone } from '@angular/core';
2
3  @Component({ selector: 'app-scroll', template: `<div>Scroll Demo</div>` })
4  export class ScrollComponent {
5      constructor(private ngZone: NgZone) {
6          this.ngZone.runOutsideAngular(() => {
7              window.addEventListener('scroll', this.onScroll);
8          });
9      }
10
11     onScroll() {
12         console.log('Scrolling...');
13     }
14 }
```

This **reduces unnecessary change detection cycles**, improving performance in heavy or frequently updating scenarios.

Angular scenario-based interview questions

These Angular scenario-based interview questions for experienced professionals test problem-solving in real-world situations.

1. Performance issue: The app is slow due to frequent change detection. How do you debug and fix this?

When an Angular app becomes slow because of frequent change detection, you can **debug** and **optimize** using the following steps.

1. Debugging

- **Angular DevTools:** Check which components trigger many change detection cycles
- **Chrome Performance Tab:** Profile JS execution and rendering
- **Look for patterns:** Large lists, heavy computations in templates, frequent event emissions (scroll, input)

2. Fixing / Optimization

1. Use OnPush Change Detection

```
1  @Component({
2    selector: 'app-list',
3    templateUrl: './list.component.html',
4    changeDetection: ChangeDetectionStrategy.OnPush,
5  })
6  export class ListComponent {}
```

- Component only checks for changes when inputs or signals change.

2. Implement trackBy in *ngFor

```
<div *ngFor="let item of items; trackBy: trackById">{{ item.name }}</div>
```

- Prevents unnecessary DOM updates by tracking items by unique identifiers

3. runOutsideAngular()

- For high-frequency events like scroll or mousemove:

```
this.ngZone.runOutsideAngular(() => {  
  window.addEventListener('scroll', this.onScroll);  
});
```

4. Move heavy computations to pipes or services

- Avoid calculations directly in templates

5. Lazy load modules and components

- Reduce initial bundle size and unnecessary checks

2. Migration: You're migrating an Angular 12 app using RxJS to Signals. How would you plan it?

Migrating from RxJS to Angular Signals requires careful planning to maintain **reactivity and performance** while reducing boilerplate.

1. Audit Current App

- Identify all **RxJS streams** (BehaviorSubject , Observable , Subject) in components and services
- Determine which streams are **local state** vs. **async external data** (HTTP, WebSocket)

2. Decide Scope

- **Local component state** → convert to **signals**
- **Service-level shared state** → consider using **signal stores** or keep RxJS if complex async flows are involved

3. Refactor Local State

- Replace BehaviorSubject / Observable used for UI state with **signals**:

```
1 import { signal } from '@angular/core';
```

```
2
3  export class CounterComponent {
4    count = signal(0);
5
6    increment() {
7      this.count.set(this.count() + 1);
8    }
9  }
```

4. Convert Derived State

- Use computed signals for derived or calculated values:

```
import { computed } from '@angular/core';

total = computed(() => this.items().reduce((sum, i) => sum + i.value, 0));
```

5. Replace Subscriptions

- Remove `.subscribe()` in templates and components
- Use **effects** or **computed signals** to react to state changes

```
1  import { effect } from '@angular/core';
2
3  effect(() => {
4    console.log('Count changed:', this.count());
5  });
```

6. Keep complex Async as RxJS (if needed)

- For multi-step streams, operators like `mergeMap` , `combineLatest` , keep RxJS to avoid complex refactors
- Gradually migrate where simpler

7. Test thoroughly

- Ensure UI updates correctly after signal migration
- Verify performance improvements and no memory leaks

3. State management: How would you architect a shared dashboard state using services or NgRx?

You can manage shared state using **services with RxJS/Signals** or **NgRx** depending on app complexity.

1. Using a Shared Service

- Simple and lightweight for **small to medium apps**.
- Use `BehaviorSubject` / `signal` to hold state and provide getters/setters.

```

1  @Injectable({ providedIn: 'root' })
2  export class DashboardService {
3      // RxJS
4      private widgetsSubject = new BehaviorSubject<Widget[]>([]);
5      widgets$ = this.widgetsSubject.asObservable();
6
7      updateWidgets(widgets: Widget[]) {
8          this.widgetsSubject.next(widgets);
9      }
10
11     // Signals (Angular 16+)
12     widgets = signal<Widget[]>([]);
13 }

```

Usage in components:

```

this.dashboardService.widgets$.subscribe(widgets => { ... });
// or
this.dashboardService.widgets.set(newWidgets);

```

2. Using NgRx Store

- Best for large apps with complex state and multiple actions
- Define actions, reducers, selectors for dashboard state

```
1  // Action
2  export const loadWidgets = createAction('[Dashboard] Load Widgets');
3
4  // Reducer
5  export const dashboardReducer = createReducer(
6    initialState,
7    on(loadWidgets, (state) => ({ ...state, loading: true })),
8  );
9
10 // Selector
11 export const selectWidgets = (state: AppState) => state.dashboard.widgets;
```

Components: Subscribe via `store.select(selectWidgets)` and dispatch actions to update state.

4. Security: Design a system to prevent unauthorized access using route guards and interceptors.

1. Route Guards

- Use `CanActivate` to **restrict routes** based on authentication or roles

```
1  @Injectable({ providedIn: 'root' })
2  export class AuthGuard implements CanActivate {
3    constructor(
4      private auth: AuthService,
5      private router: Router,
6    ) {}
7    canActivate(): boolean {
8      if (this.auth.isLoggedIn()) return true;
9      this.router.navigate(['/login']);
```

```
10     return false;
11   }
12 }
```

Usage in routing module:

```
{ path: 'dashboard', component: DashboardComponent, canActivate: [AuthGuard] }
```

2. HTTP Interceptors

- Automatically attach tokens and handle unauthorized responses

```
1  @Injectable()
2  export class AuthInterceptor implements HttpInterceptor {
3    constructor(private auth: AuthService) {}
4    intercept(req: HttpRequest<any>, next: HttpHandler) {
5      const token = this.auth.getToken();
6      const authReq = req.clone({
7        setHeaders: { Authorization: `Bearer ${token}` },
8      });
9      return next.handle(authReq);
10    }
11  }
```

Register interceptor:

```
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
];
```

5. Build Time Optimization: The production build is 3MB; what steps do you take to reduce it?

If your Angular app's production build is large (e.g., 3MB), you can optimize it using these strategies:

1. Lazy Loading

- Split modules and load them only when needed.

```
{ path: 'dashboard', loadChildren: () => import('./dashboard/dashboard.module').then(m => r
```



2. Remove Unused Code

- Use tree-shakable providers and remove unused libraries or components

3. Enable Build Optimizations

```
ng build --prod --optimization --build-optimizer
```

4. Minify & Compress Assets

- Enable Terser for JS and gzip/brotli for server delivery

5. Use OnPush & Signals

- Optimize change detection to avoid heavy runtime processing

6. Externalize Large Libraries

- Load heavy libraries (e.g., lodash, moment) via **CDN** or import only required functions

Combining lazy loading, tree-shaking, and asset optimization can significantly reduce bundle size.

6. SEO & SSR: You're asked to add Server-Side Rendering (SSR) to an existing Angular SPA. How would you do it?

To add Server-Side Rendering (SSR) to an existing Angular Single Page Application (SPA), I would use Angular Universal, which enables SSR for Angular apps. The steps are:

1. **Add Angular Universal:** Use the Angular CLI to add SSR support:

```
ng add @nguniversal/express-engine
```

This sets up a server-side app module (`app.server.module.ts`) and configures an Express server (`server.ts`) to render Angular on the server.

2. **Update App for SSR Compatibility:**

- Ensure that browser-specific APIs like `window` , `document` , and `localStorage` - are only used in the browser context
- Wrap such code using `isPlatformBrowser` from `@angular/common`

3. **Build and Serve SSR:** Build both client and server bundles:

```
npm run build:ssr  
npm run serve:ssr
```

This serves the pre-rendered HTML from the server, improving SEO and initial load performance.

4. **Optional Enhancements:**

- Implement lazy loading and pre-rendering for critical routes to boost SEO
- Add meta tags dynamically using `Meta` and `Title` services for better indexing

Result: Users and search engines receive fully rendered HTML on the first request, improving SEO, performance, and crawlability of the Angular app.

7. Modularization: How do you break down a monolithic Angular app into feature modules?

To break down a monolithic Angular app into feature modules, I would follow these steps:

1. **Identify Features:** Analyze the app and group related functionality (components, services, and routes) into logical features like `UserModule` , `DashboardModule` , `ProductsModule` , etc.
2. **Create Feature Modules:** Use the Angular CLI or manually create modules:

```
ng generate module feature-name --route feature-name --module app.module
```

This sets up lazy-loaded feature modules if needed.

3. **Move Components, Services, and Routing:**

- Move all components, directives, and pipes related to the feature into the module
- Move feature-specific services into the module (consider `providedIn: FeatureModule` if appropriate)
- Create a feature routing module (`feature-name-routing.module.ts`) to handle internal routes

4. **Lazy Loading:** Update the main app routing to lazy load feature modules:

```
{ path: 'dashboard', loadChildren: () => import('./dashboard/dashboard.module').then(m => r
```

5. **Shared and Core Modules:**

- Extract reusable components, directives, and pipes into a `SharedModule`
- Keep singleton services in a `CoreModule` imported only once in `AppModule`

6. **Test and Refactor:** Ensure each module works independently and routes/services are correctly wired. This improves maintainability, code reusability, and enables lazy loading for better performance

8. Testing strategy: How do you structure unit tests for complex component hierarchies?

When structuring unit tests for complex component hierarchies, I follow these practices:

1. Test Components in Isolation:

- Use Angular's `TestBed` to create a testing module for the component
- Mock child components, directives, and services to isolate the component under test

2. Use Shallow Testing for Parents:

- Replace child components with stubs or mocks to focus on parent component behavior without testing children's internal logic

3. Test Inputs, Outputs, and DOM Interactions:

- Verify `@Input()` bindings, `@Output()` events, template rendering, and user interactions
- Ensure component reacts correctly to input changes and emits expected events

4. Service Dependencies:

- Use spies or mock services for any external dependencies to control behavior and avoid side effects

5. Organize Tests Logically:

- Group tests by functionality (rendering, events, service calls) using `describe` blocks
- Keep tests small, focused, and maintainable, mirroring the component's structure

Result: This approach ensures each component's behavior is tested reliably while keeping tests fast, maintainable, and isolated from unrelated parts of the hierarchy.

Advanced Angular interview questions

These questions push beyond fundamentals - they're frequent in senior developer and tech lead interviews.

1. How does the Angular DI lifecycle differ across lazy-loaded and eagerly-loaded modules?

Angular creates a single root injector for eagerly-loaded code, while each lazy-loaded route gets its own child injector.

- Eager providers (AppModule) → singletons app-wide
- Lazy module providers → new instance per lazy boundary (scoped to that route tree)
- `providedIn: 'root'` → one instance in root; `providedIn: 'any'` → one per injector (root and each lazy)

```
1  @Injectable({ providedIn: 'any' })
2  export class FeatureService {}
3
4  // Eager context and each lazy module will get different instances
5  constructor(private s: FeatureService) {}
```

This scoping helps isolate features and avoid cross-feature state leaks while keeping true singletons in root.

2. Explain hydration in Angular and how partial rehydration improves SSR speed.

Hydration attaches Angular runtime to server-rendered HTML without re-rendering it. The DOM remains; Angular wires up event listeners and state.

- Faster TTI: skip initial client re-render
- Fewer layout thrashes vs CSR
- Partial hydration (deferred/fragment hydration) hydrates only critical views first and defers the rest until visible or interacted

```
1  // main.ts (standalone app)
2  bootstrapApplication(AppComponent, {
3    providers: [provideClientHydration()],
```

```
4  });  
5  
6  // Template: defer non-critical islands to reduce hydration cost  
7  @defer (on viewport) {  
8    <heavy-widget />  
9  } @placeholder { Loading... }
```

3. How does standalone component architecture simplify Angular dependency graphs?

Standalone components remove NgModule indirection—dependencies are imported where used, and providers are scoped closer to usage.

- Fewer module graphs to reason about; imports are explicit per component
- Better tree-shaking; simpler incremental adoption
- Component-level providers reduce accidental singletons

```
1  @Component({  
2    standalone: true,  
3    selector: 'user-card',  
4    imports: [CommonModule],  
5    template: `{{ user.name }}`,  
6    providers: [UserLocalLogger],  
7  })  
8  export class UserCard {  
9    @Input() user!: User;  
10 }  
11  
12 bootstrapApplication(AppComponent, {  
13   providers: [provideRouter(routes)],  
14 });
```

4. What are the differences between zone-full and zone-less change detection setups?

- Zone-full (default): Zone.js patches async APIs and triggers global change detection automatically. Simpler dev ergonomics; higher runtime overhead
- Zone-less: Disable Zone.js and use push-based patterns (Signals, OnPush, manual triggers). Lower overhead; you opt in to updates

```

1  // Zoneless bootstrap (no global auto change detection)
2  bootstrapApplication(AppComponent, { ngZone: 'noop' });
3
4  @Component({ changeDetection: ChangeDetectionStrategy.OnPush })
5  export class Cmp {
6    // Prefer Signals or async pipe; call cdr.markForCheck() when bridging imperative co
7    constructor(private cdr: ChangeDetectorRef) {}
8  }

```

Use zoneless with Signals and `async pipe` for most UI; use `runOutsideAngular()` for high-frequency events.

5. Explain how to implement custom preloading strategies in routing.

Implement `PreloadingStrategy` to decide per-route preloading (e.g., based on flags or network conditions).

```

1  @Injectable({ providedIn: 'root' })
2  export class SelectivePreloading implements PreloadingStrategy {
3    preload(route: Route, load: () => Observable<any>) {
4      return route.data?.['preload'] ? load() : of(null);
5    }
6  }
7
8  const routes: Routes = [
9    {
10     path: 'admin',
11     loadChildren: () => import('./admin/admin.routes'),
12     data: { preload: true },
13   },
14   { path: 'reports', loadChildren: () => import('./reports/reports.routes') },

```

```
15   ];  
16  
17   provideRouter(routes, withPreloading(SelectivePreloading));
```

You can enrich logic with `navigator.connection`, user roles, or time-based delays.

6. How do you leverage the build optimizer, budgets, and source-map-analyzer to cut bundle sizes?

- Build optimizer: enabled by default in production; ensures better tree-shaking and Terser minification
- Budgets: enforce caps to catch regressions during CI
- Source map analysis: find large deps and heavy modules

```
1  // angular.json (excerpt)  
2  {  
3    "configurations": {  
4      "production": {  
5        "budgets": [  
6          { "type": "initial", "maximumWarning": "500kb", "maximumError": "2mb" },  
7          { "type": "anyComponentStyle", "maximumWarning": "150kb" }  
8        ]  
9      }  
10   }  
11 }
```

Workflow:

1. Build with stats: `ng build --configuration production --stats-json`
2. Analyze: `npx source-map-explorer dist/**/*.js` (or webpack bundle analyzer)
3. Act: lazy-load, split routes, replace heavy libs (e.g., date-fns over moment), use `providedIn: 'root'` / tree-shakable APIs

7. What Common RxJS pitfalls (like unhandled subscriptions) can cripple Angular scalability?

- Leaks from manual `subscribe()` without `unsubscribe` → prefer `async pipe` or `takeUntilDestroyed()`
- Nested subscribes → use flattening operators (`switchMap` , `concatMap`)
- Re-executed HTTP on every subscription → share results (`shareReplay({ refCount: true, bufferSize: 1 })`)
- Missing error handling → `catchError` with fallbacks

```

1  import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
2
3  this.form.valueChanges
4    .pipe(
5      debounceTime(300),
6      switchMap((v) => this.api.search(v)),
7      takeUntilDestroyed(this.destroyRef),
8    )
9    .subscribe((result) => this.results.set(result));

```

Prefer `async pipe` in templates; for shared streams, cache with `shareReplay(1)` .

8. Describe how to implement CI/CD for an Angular app (testing, linting, SSR build, deployment pipelines).

Core stages: install → lint → test → build (SPA/SSR) → artifact → deploy.

```

1  # .github/workflows/ci.yml (simplified)
2  name: Angular CI
3  on: [push, pull_request]
4  jobs:
5    build:
6      runs-on: ubuntu-latest
7      steps:

```

```
8      - uses: actions/checkout@v4
9      - uses: actions/setup-node@v4
10     with: { node-version: 20, cache: npm }
11     - run: npm ci
12     - run: npm run lint
13     - run: npm test -- --watch=false --browsers=ChromeHeadless
14     - run: npm run build:ssr # e.g., ng build && ng run app:server
15     - uses: actions/upload-artifact@v4
16     with: { name: dist, path: dist/ }
17
18 # Deploy job can download artifact and push to hosting (Firebase, Vercel, Azure, etc.)
```

Include budgets in CI to block regressions; for SSR, validate hydration with e2e smoke tests before deploy.

Best practices & optimization tips

- Use **OnPush** strategy for pure components
- Track lists using `trackBy` in `*ngFor` to minimize DOM re-renders
- Use **Signals** for local reactive patterns; keep global state in NgRx or Akita
- Always unsubscribe or leverage `async pipe` or `takeUntilDestroyed()`
- Avoid deep component hierarchies; use standalone components where possible
- Compress assets and enable **AOT Compilation** for production builds
- Use `Renderer2` for cross-platform DOM-safe operations
- Keep the codebase modular with feature-based folder organization

Following these keeps senior developers' Angular apps maintainable and performant in large-scale setups

Continue Learning

Here are a few resources to expand your prep:

- [Angular Interview Questions for Beginners](#)
 - [Angular UI interview questions at GreatFrontEnd](#)
-

Angular has matured rapidly with standalone APIs, fine-grained reactivity using Signals, and better SSR tooling. Interviewers now test practical knowledge - not syntax, but design reasoning and architectural excellence.

If your goal is to ace **Angular Experienced Interview Questions**, practice code samples, profile performance, and be ready to justify your design choices like a senior engineer.

Ready to practice with real Angular interview questions?

Level up your Angular interview prep with our carefully curated collection of [Angular interview questions at GreatFrontEnd](#). Practice real UI questions from top tech companies and compare your approach with official solutions from experts.

[🔗 Copy link](#)[🔗 Share](#)

← Previous

50 React.js Interview Questions for Experienced Developers

Subscribe to our newsletter

[Notify me](#)

Practice

[Get started](#)[JavaScript functions](#)[User interface coding](#)[System design](#)[Quiz](#)

Guides

[Front End Interview Playbook](#)[Front End System Design Playbook](#)[React Interview Playbook](#)[Behavioral Interview Playbook](#)

Study Plans

[1 Week Plan](#)[1 Month Plan](#)[3 Months Plan](#)

Company

[Pricing](#)[Promotions](#)[Roadmap](#)[About](#)[Team](#)[Contact us](#)[Advertise with us](#) New[Become an affiliate](#)[Careers](#)[Blog](#)[Medium](#)[DEV Community](#)[English \(US\) ▾](#)

© 2026 Codeney Pte Ltd. All rights reserved.

[Privacy Policy](#)[Terms of Service](#)